



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

ОТЧЕТ

по лабораторной работе №5

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация обратного индекса»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	Обратный индекс	3
1.2	Операции над posting lists	3
1.3	Skip lists	3
1.4	Сжатие posting lists	4
1.5	Ранжирование BM25	4
2	Практическая часть	5
2.1	Архитектура реализации	5
2.2	Формат хранения индекса	6
2.3	Поиск	6
3	Исследовательская часть	7
3.1	Аппаратные характеристики	7
3.2	Методика замеров	7
3.3	Результаты построения индекса	8
3.4	Результаты wairmir	9
3.5	Результаты поиска в индексе	10
3.6	Результаты поиска с ранжированием	13
3.7	Размер индекса	15
3.8	Сжатие индекса	16
3.9	Профилирование	18
4	Вывод	22

1 Теоретическая часть

1.1 Обратный индекс

Обратный индекс (inverted index) — структура данных, отображающая термы в список документов (posting list), в которых они встречаются. Для каждого терма хранится упорядоченный по DocID список постингов, содержащих идентификатор документа и позиции вхождения терма. Обратный индекс является основной структурой данных в поисковых системах, обеспечивая быстрый поиск документов по ключевым словам.

Формат постинга: $(DocID, [pos_1, pos_2, \dots, pos_k])$, где $DocID$ — идентификатор документа, pos_i — позиции вхождения терма в документе. Позиции необходимы для реализации операторов (ADJ, NEAR).

1.2 Операции над posting lists

Основные операции над posting lists, используемые в поисковых запросах:

- *Intersect* $(A \cap B)$ — пересечение документов (оператор AND);
- *Union* $(A \cup B)$ — объединение документов (оператор OR);
- *Difference* $(A \setminus B)$ — разность документов (оператор NOT);
- *Adjacent* — документы, где термы стоят подряд (оператор ADJ);
- *Near* — документы, где термы стоят на расстоянии $\leq k$ (оператор NEAR/ k).

1.3 Skip lists

Skip list — вероятностная структура данных поверх упорядоченного списка, позволяющая выполнять поиск за $O(\log(n))$ вместо $O(n)$. В данной реализации skip list строится детерминированно: на уровне i размещаются элементы с шагом 2^{i+1} , начиная с $\lceil \sqrt{n} \rceil$. Итератор PostingIterator поддерживает курсоры на каждом уровне и метод AdvanceTo(docID), который перемещает итератор к первому элементу $\geq docID$, используя skip-указатели для быстрого продвижения.

1.4 Сжатие posting lists

Если длина массива ≥ 32 , используется поблочный битпакинг (block size 32), где каждый блок из 32 дельт кодируется минимальным числом бит; иначе применяется varint-кодирование (`binary.PutUvarint`). Битпакинг обеспечивает сжатие в 3–4 раза по сравнению с varint за счёт хранения фиксированного числа бит на элемент вместо переменного.

1.5 Ранжирование BM25

BM25 — функция ранжирования, вычисляющая релевантность документа запросу:

$$\text{score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl})}$$

, где $f(t, D)$ — частота термина t в документе D , $|D|$ — длина документа, avgdl — средняя длина документа, $k_1 = 1.5$ и $b = 0.75$ — параметры.

$$\text{IDF}(t) = \ln(1 + (N - n(t) + 0.5) / (n(t) + 0.5))$$

, где N — число документов, $n(t)$ — число документов, содержащих терм t .

2 Практическая часть

2.1 Архитектура реализации

Проект реализован на Go и состоит из следующих пакетов:

1) `internal/index`:

- `IndexBuilder` — построение индекса, добавление документов, токенизация, построение posting lists, дельта-кодирование и битпакинг;
- `InvertedIndex` — загрузка индекса через `mmap`, декодирование posting list;
- `PostingList` — постинг-лист со skip list и итератором `PostingIterator`;
- операции `Intersect`, `Union`, `Difference`, `Adjacent`, `Near`.

2) `internal/query`:

- `Parser` — рекурсивный парсинг запроса с операторами `AND`, `OR`, `NOT`, `ADJ`, `NEAR/k` и скобками;
- `Evaluator` — обход AST и вызов соответствующих операций над posting lists.

3) `internal/ranking` — ранжирование BM25;

4) `internal/compression` — битпакинг и varint-кодирование/декодирование posting lists;

5) `internal/storage` — двоичный формат индекса (header + term entries + postings data) и хранилище документов (`DocStore`);

6) `internal/engine` — поисковый движок, объединяющий все необходимые компоненты;

7) `cmd/tui` — TUI-интерфейс на `bubbletea`.

2.2 Формат хранения индекса

Индекс хранится в двух файлах:

- `*.idx` — сам индекс с заголовком (40 байт: magic, версия, число термов, число документов, число токенов, смещения), отсортированными term entries и сжатыми posting lists;
- `*.docs` — хранилище документов.

Обратный индекс загружается через `mmap` и читает только заголовки и term entries, а posting lists декодируются лениво при поиске. Страницы `mmap` подгружаются операционной системой при первом обращении. Для каждого терма posting list хранится в виде плоского массива: $[len_{docDeltas}][docDeltas][len_{posDeltas}][posDeltas][len_{tfs}][tfs][len_{skipList}][skipList]$, где каждый подмассив сжимается независимо (битпакинг или varint в зависимости от размера), а затем весь блок кодируется одним вызовом `Compress`.

2.3 Поиск

Обработка запроса состоит из трёх этапов:

- 1) *Парсинг*: строка запроса разбирается в AST;
- 2) *Вычисление*: обход AST, для каждого терма загружается posting list, операции выполняются в соответствии с приоритетом слева направо;
- 3) *Ранжирование*: для документов из результирующего posting list вычисляется BM25 score.

3 Исследовательская часть

3.1 Аппаратные характеристики

Все замеры проводились на ноутбуке со следующей конфигурацией:

- операционная система: macOS (arm64);
- процессор: Apple M3 Pro, 11 ядер;
- оперативная память: 18 ГБ;
- версия Go: go1.25.

Во время замеров не выполнялись ресурсоёмкие фоновые задачи, а ноутбук был подключён к сети электропитания.

3.2 Методика замеров

Для замеров использованы встроенные benchmark-тесты Go. Измерялись четыре операции:

- **build** — построение индекса;
- **warmup** — прогрев индекса;
- **search** — поиск по запросам различных типов (без ранжирования);
- **searchrank** — поиск с ранжированием BM25.

Для построения индекса использованы документы размером 500, 700, 1000, 2000, 3000, 4000, для поиска и warmup — документы размером 10000, 25000, 50000, 75000, 100000, 125000, 150000, 200000. Разделение размеров обусловлено тем, что построение индекса при больших N не успевает выполнить более одной итерации за таймаут.

Для каждого размера фиксировались:

- среднее время на операцию (нс/оп);
- 95% доверительный интервал (CI95);
- размер индекса в байтах (для build).

Поиск измеряется для следующих типов запросов: **Term**, **And**, **Or**, **Not**, **Adj**, **Near**, а также составных: **Complex_AndOr**, **Complex_AdjAnd**, **Complex_AndNot**. Для каждого типа генерируется 10000 случайных запросов. В качестве термов используются термы из нижней половины распределения по частоте документа, что позволяет избежать аномально медленных запросов на высокочастотных термах. Результатом является среднее время на один запрос.

3.3 Результаты построения индекса

Таблица 3.1 содержит результаты замеров времени построения индекса и его размера.

Таблица 3.1 – Время построения индекса и размер

N	время (мс)	размер (МБ)
500	281 ± 14	5
700	334 ± 13	7
1000	456 ± 4	9
2000	851 ± 44	16
3000	1206 ± 27	23
4000	1623 ± 7	29

График времени построения индекса представлен на рисунке 3.1.

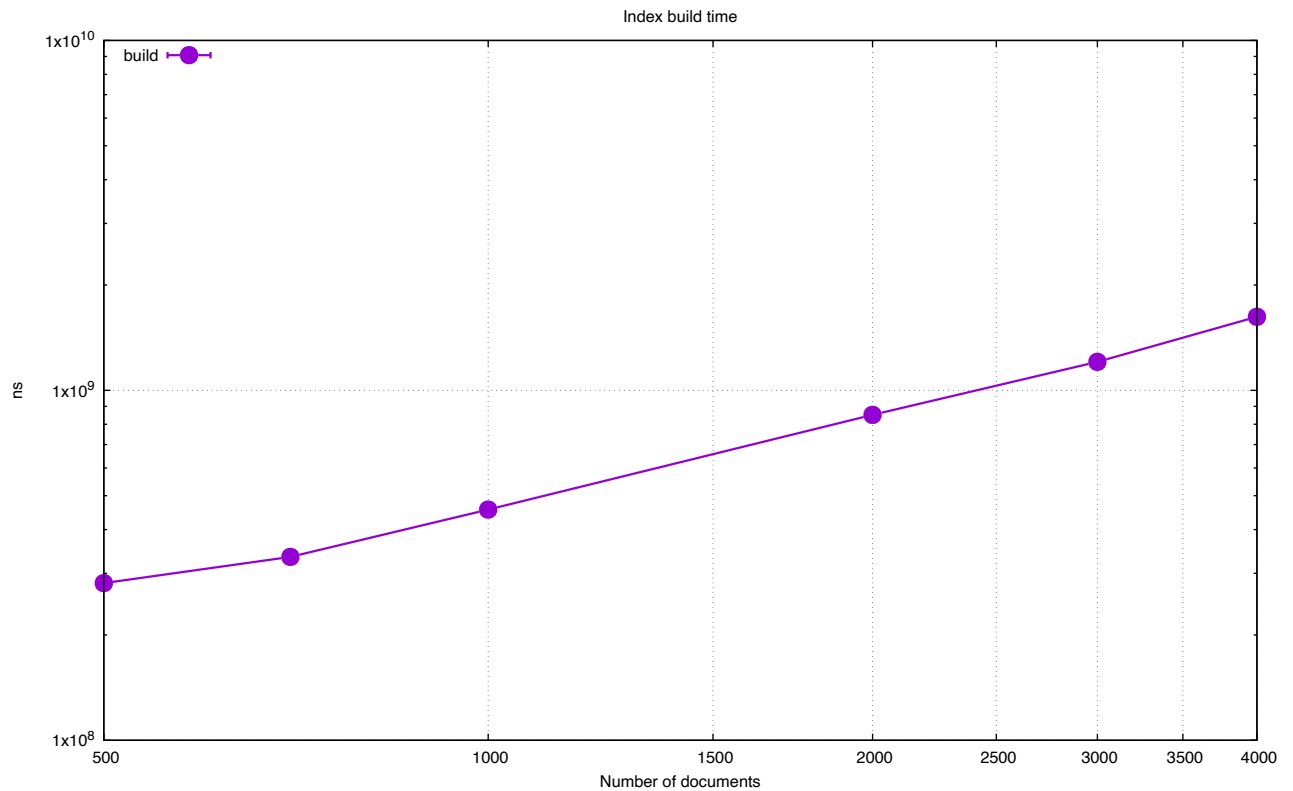


Рисунок 3.1 – Время построения индекса в зависимости от числа документов

Время построения растёт линейно. При увеличении N в 8 раз (от 500 до 4000) время увеличивается примерно в 6 раз (от 281 до 1623 мс).

3.4 Результаты warmup

Таблица 3.2 содержит результаты замеров прогрева индекса.

Таблица 3.2 – Время warmup (мс)

N	время (мс)
10000	129 ± 16
25000	366 ± 23
50000	644 ± 13
75000	744 ± 60
100000	823 ± 8
125000	852 ± 32
150000	1097 ± 15
200000	1337 ± 42

График времени warmup представлен на рисунке 3.2.

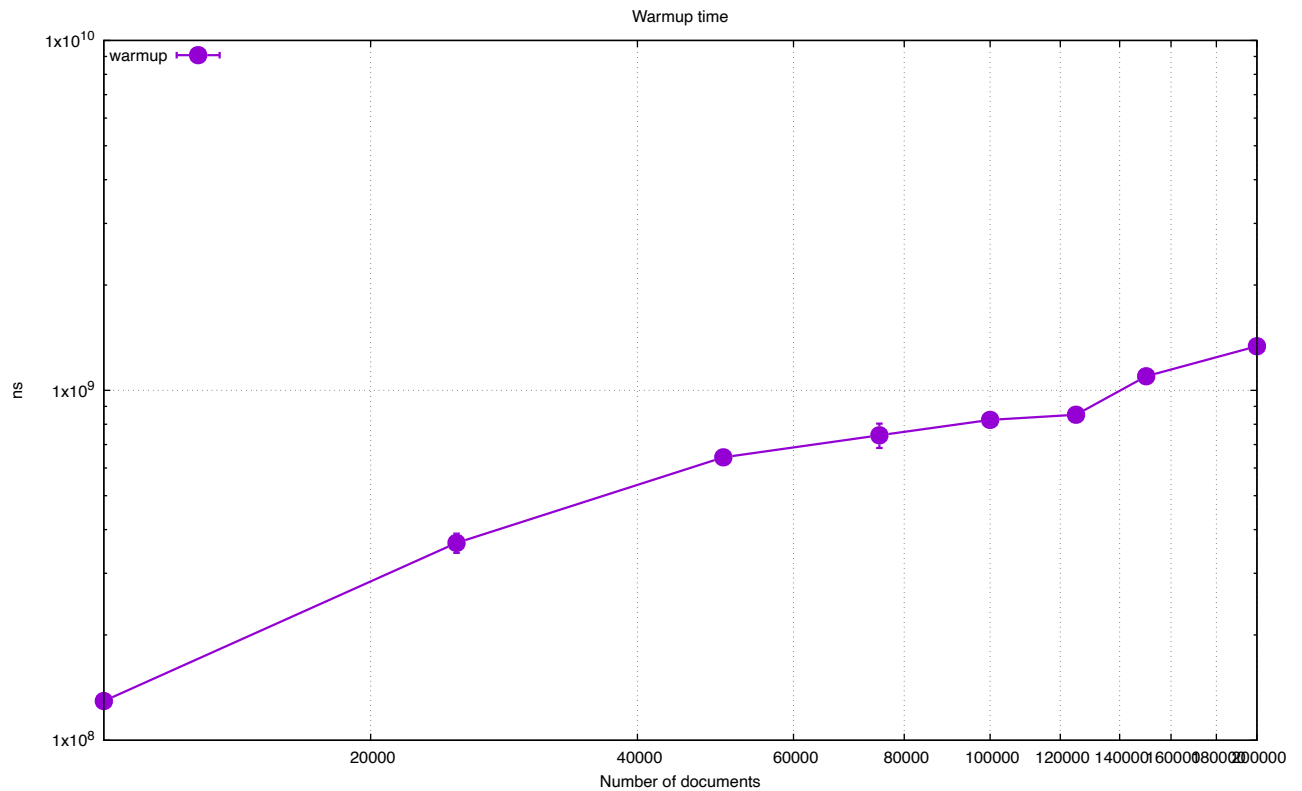


Рисунок 3.2 – Время первого запроса после загрузки индекса (warmup)

Время warmup значительно превышает время повторного поиска. Это объясняется тем, что при первом обращении к posting lists операционная система подгружает страницы из диска в память. С ростом N время warmup растёт, что объясняется увеличением числа mmap-страниц, к которым обращается поисковый запрос.

3.5 Результаты поиска в индексе

Таблица 3.3 содержит результаты замеров времени базовых операций поиска.

Таблица 3.3 – Время базовых операций поиска (нс)

N	Term	And	Or	Not	Adj	Near
10000	2771 ± 3	5821 ± 5	6073 ± 5	6315 ± 5	6004 ± 5	6138 ± 5
25000	2876 ± 3	5943 ± 5	6413 ± 7	6398 ± 5	6105 ± 5	6276 ± 6
50000	2916 ± 3	6063 ± 5	7135 ± 58	6461 ± 6	6213 ± 5	6324 ± 6
75000	2926 ± 3	6179 ± 6	10385 ± 137	6548 ± 5	6262 ± 6	6394 ± 6
100000	2952 ± 3	6514 ± 7	11700 ± 398	6660 ± 6	6329 ± 6	6426 ± 6
125000	3064 ± 3	6512 ± 7	12832 ± 884	6720 ± 6	6383 ± 6	6485 ± 6
150000	3055 ± 3	7041 ± 9	16720 ± 383	6911 ± 6	6504 ± 6	6702 ± 6
200000	3143 ± 3	7134 ± 9	26040 ± 802	6856 ± 6	6602 ± 6	6767 ± 6

Таблица 3.4 содержит результаты замеров времени составных операций поиска.

Таблица 3.4 – Время составных операций поиска (нс)

N	OrAnd	AdjAnd	AndNot
10000	12060 ± 11	9009 ± 8	12330 ± 10
25000	12335 ± 25	9220 ± 8	12536 ± 11
50000	12264 ± 10	9277 ± 8	12725 ± 11
75000	12837 ± 59	9406 ± 8	12720 ± 11
100000	12765 ± 51	9485 ± 8	12799 ± 11
125000	12923 ± 53	9592 ± 8	12866 ± 11
150000	13221 ± 77	9635 ± 8	12956 ± 11
200000	13289 ± 63	9999 ± 9	13105 ± 12

Графики времени поиска представлены на рисунках 3.3 и 3.4.

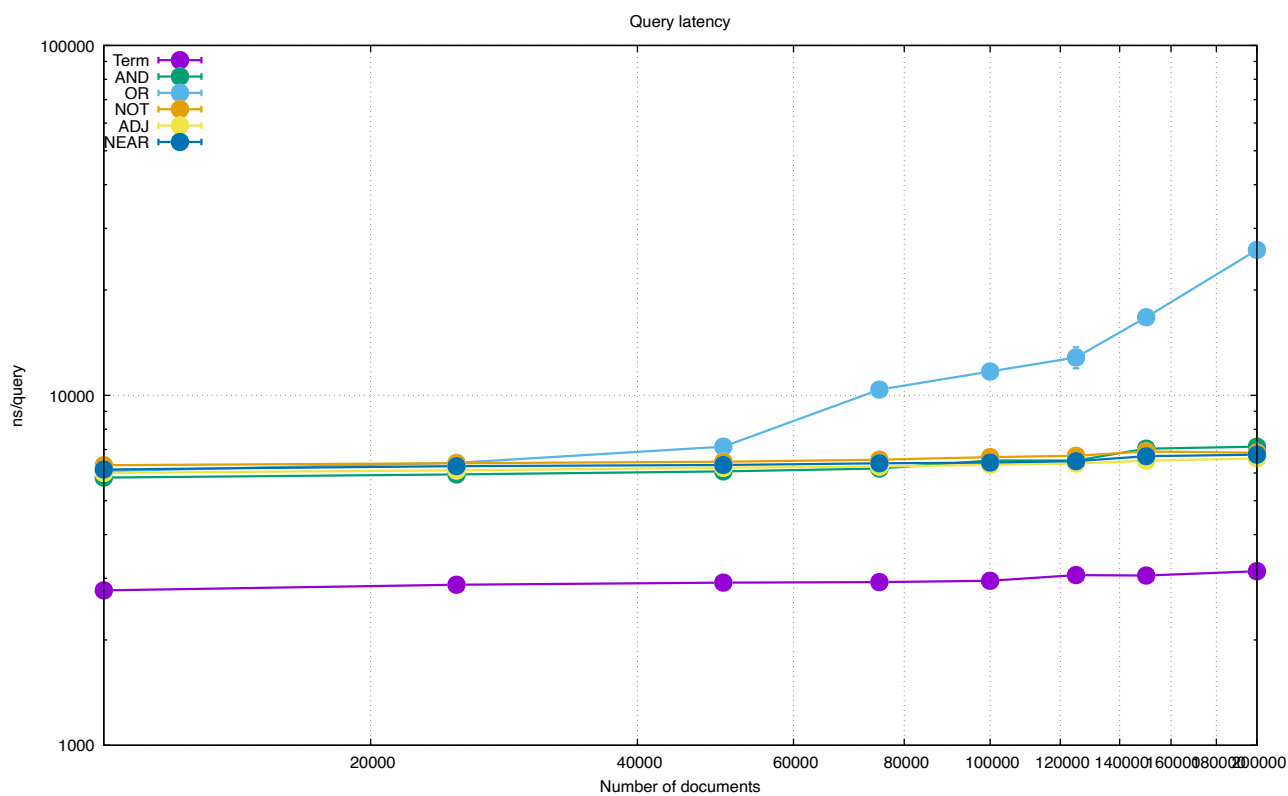


Рисунок 3.3 – Время базовых операций поиска

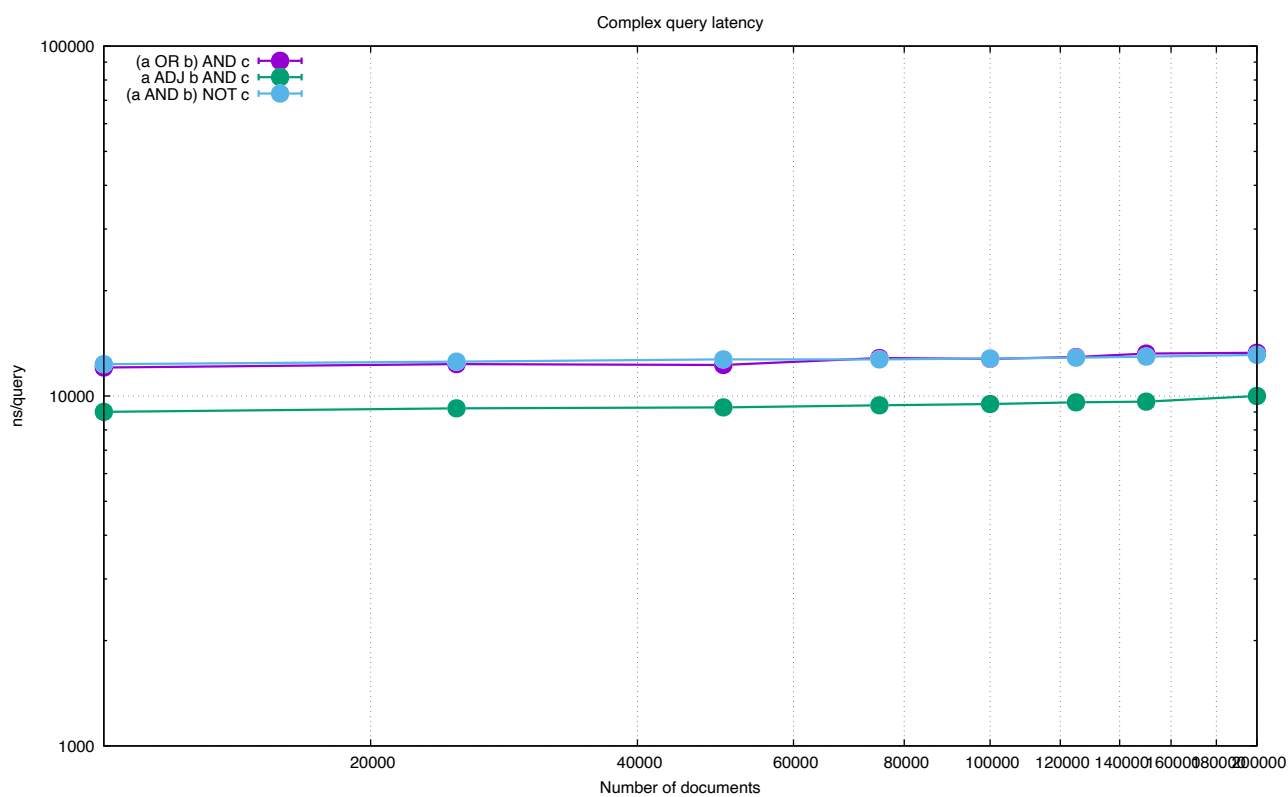


Рисунок 3.4 – Время составных операций поиска

3.6 Результаты поиска с ранжированием

Таблица 3.5 содержит результаты замеров времени базовых операций поиска с ранжированием BM25.

Таблица 3.5 – Время базовых операций поиска с ранжированием (нс)

N	Term	And	Or	Not	Adj	Near
10000	2960 ± 3	5879 ± 5	6398 ± 6	6512 ± 5	6079 ± 5	6203 ± 5
25000	3021 ± 3	6069 ± 5	6575 ± 6	6662 ± 6	6181 ± 5	6358 ± 6
50000	3082 ± 3	6185 ± 5	7165 ± 22	6740 ± 6	6256 ± 6	6433 ± 6
75000	3106 ± 3	6257 ± 6	10381 ± 485	6821 ± 6	6306 ± 6	6517 ± 6
100000	3231 ± 3	6713 ± 9	13234 ± 957	6981 ± 6	6360 ± 6	6543 ± 6
125000	3259 ± 3	6609 ± 7	14449 ± 1091	6986 ± 6	6409 ± 6	6626 ± 6
150000	3535 ± 4	7560 ± 11	20583 ± 1275	7415 ± 7	6895 ± 7	7034 ± 7
200000	3674 ± 4	8218 ± 38	28927 ± 1099	7473 ± 7	6959 ± 7	7153 ± 7

Таблица 3.6 содержит результаты замеров времени составных операций поиска с ранжированием.

Таблица 3.6 – Время составных операций поиска с ранжированием (нс)

N	OrAnd	AdjAnd	AndNot
10000	12102 ± 10	9181 ± 8	12314 ± 10
25000	12333 ± 10	9396 ± 8	12568 ± 11
50000	12511 ± 11	9506 ± 8	12688 ± 11
75000	12844 ± 48	9686 ± 8	12768 ± 11
100000	12945 ± 52	9735 ± 8	12812 ± 11
125000	13074 ± 51	9754 ± 8	12931 ± 11
150000	13725 ± 57	10238 ± 10	13455 ± 14
200000	14464 ± 83	10507 ± 11	13675 ± 14

Графики времени поиска с ранжированием BM25 представлены на рисунках 3.5 и 3.6.

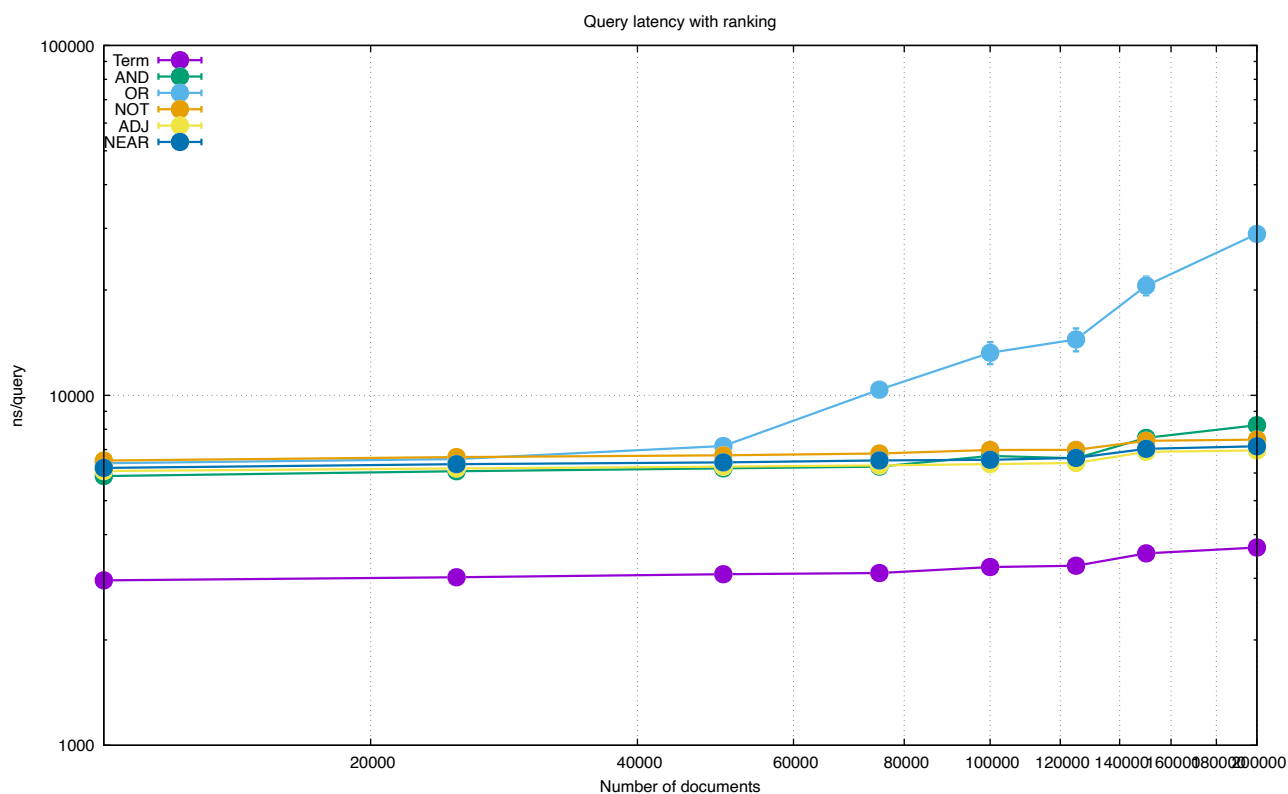


Рисунок 3.5 – Время базовых операций поиска с ранжированием

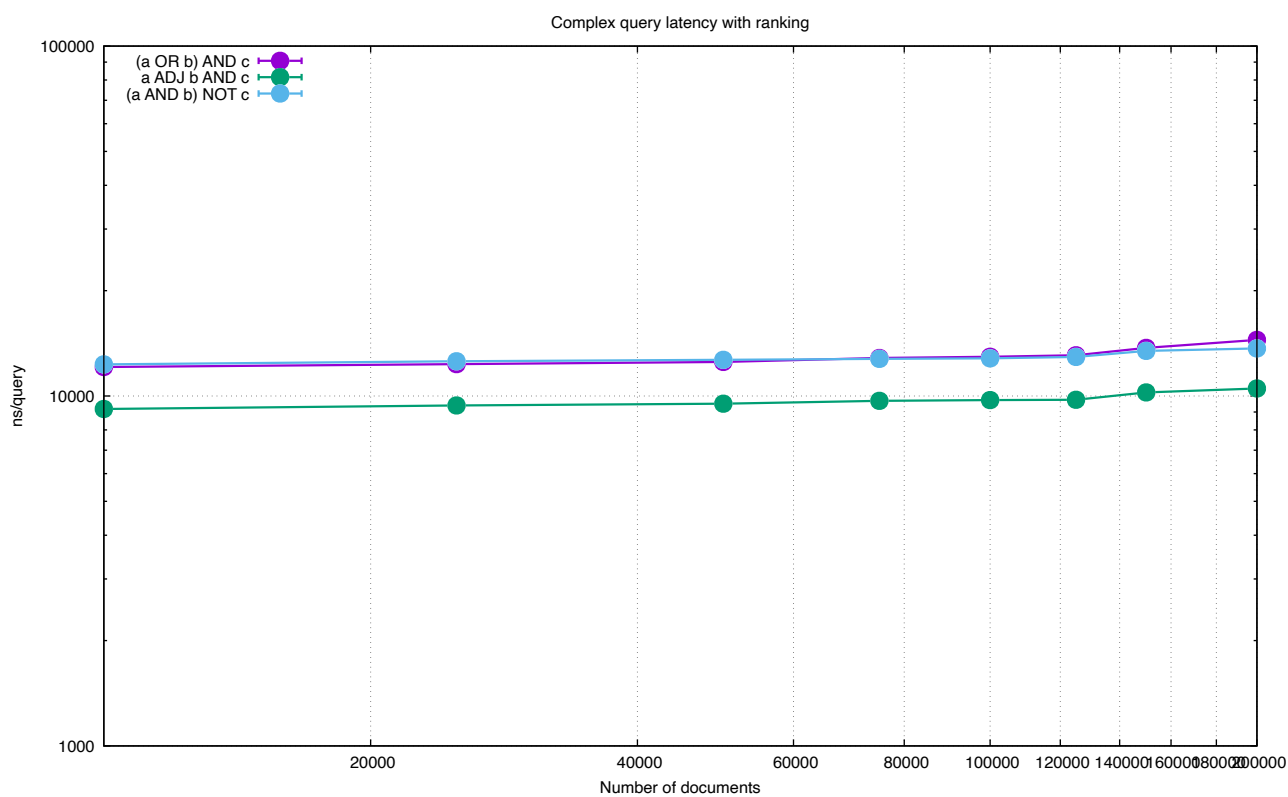


Рисунок 3.6 – Время составных операций поиска с ранжированием

Время поиска растёт с увеличением N для всех операторов, что объ-

ясняется ростом длины posting lists и, соответственно, увеличением объёма вычислений при пересечении/объединении.

Оператор **Term** является самым быстрым (≈ 3 мкс), поскольку требует лишь загрузки одного posting list. Среди бинарных операторов **And** и **Not** — самые быстрые (≈ 7 мкс при $N = 200000$), а **Or** — самый медленный (≈ 26 мкс). Операторы **Adj** и **Near** медленнее **And**.

Составные запросы показывают время, сопоставимое с суммой времён базовых операторов, что подтверждает корректность поэтапного вычисления. **Complex_OrAnd** — самый медленный составной запрос (≈ 13 мкс при $N = 200000$), что объясняется наличием операции **Or**.

3.7 Размер индекса

График размера индекса в зависимости от числа документов представлен на рисунке 3.7.

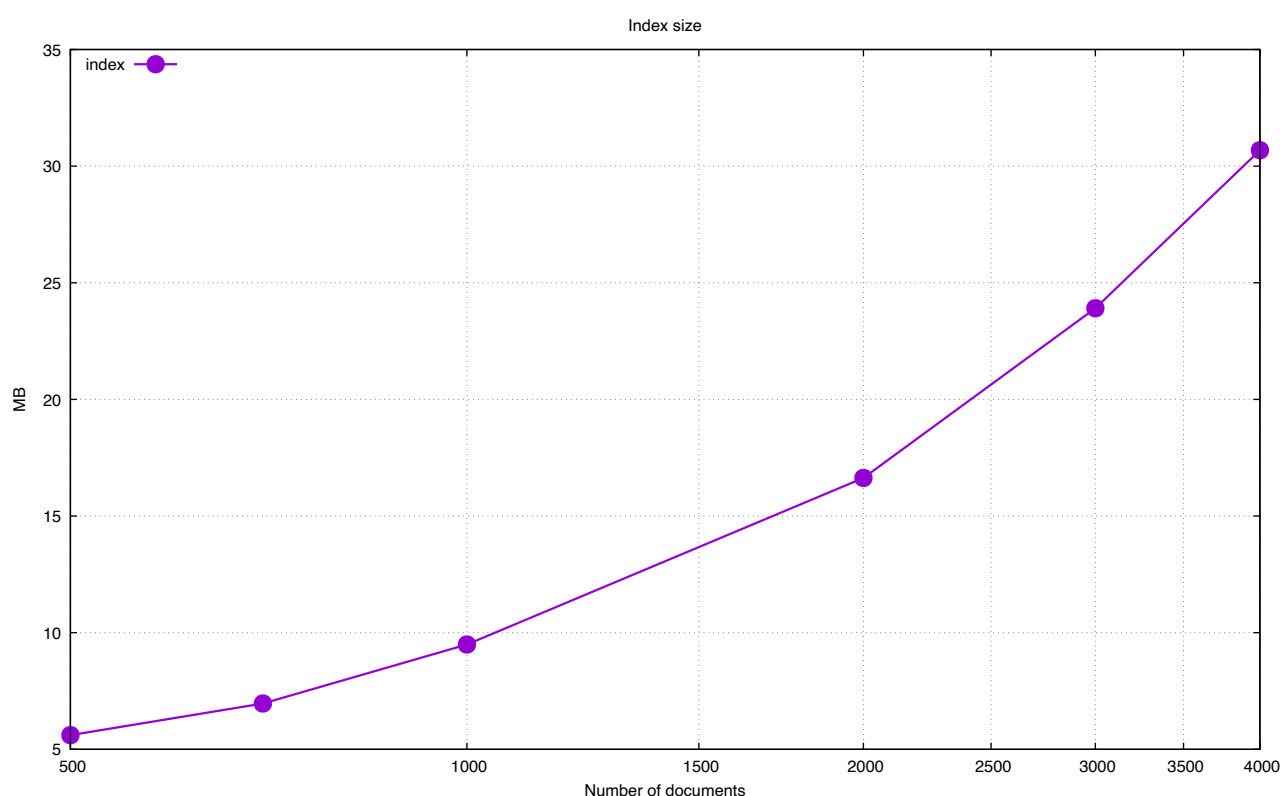


Рисунок 3.7 – Размер индекса в зависимости от числа документов

Размер индекса растёт линейно с числом документов. Это объясняется тем, что основную долю занимают posting lists и данные документов, объём которых пропорционален числу токенов.

3.8 Сжатие индекса

Для оценки эффективности сжатия сравнивается размер данных в двух форматах:

- *Raw* — несжатое представление, где каждый элемент массива `uint32` хранится как 4 байта;
- *Compressed* — сжатое представление с битпакингом (блоки по 32 элемента).

Размеры включают posting lists (`docDeltas`, `posDeltas`, `tfs`, `skipLists`) и метаданные (`docFreqs`, `offsetDeltas`, `postingsLengths`, `docLengths`), но не включают заголовок и строки термов, которые идентичны в обоих форматах.

Таблица 3.7 содержит результаты замеров сжатия индекса.

Таблица 3.7 – Сжатие индекса

N	Raw (МБ)	Compressed (МБ)	Экономия (%)	Коэффициент
10000	212	69	67	3.06
25000	423	134	68	3.15
50000	543	170	69	3.19
75000	661	205	69	3.23
100000	776	238	69	3.26
125000	893	272	70	3.28
150000	1007	305	70	3.30
200000	1235	371	70	3.33

Графики сжатия представлены на рисунках 3.8 и 3.9.

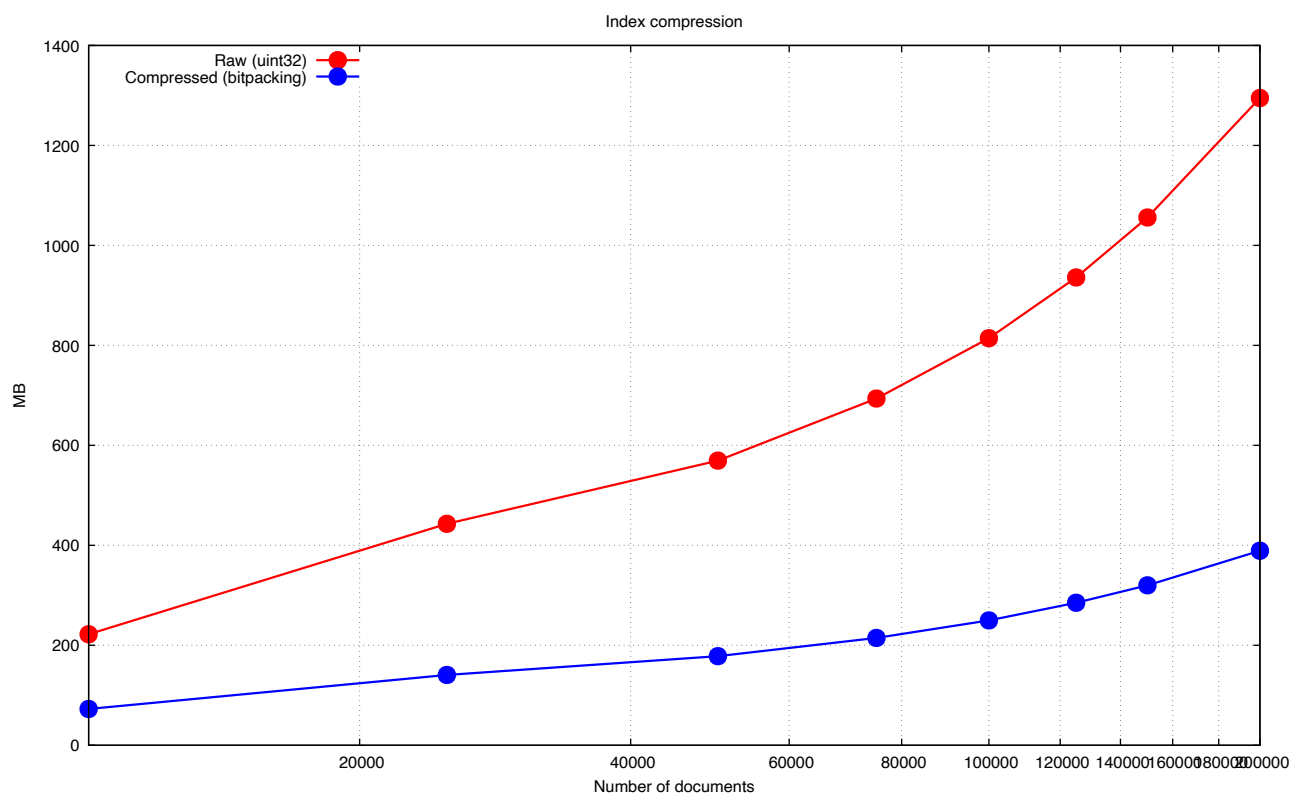


Рисунок 3.8 – Размер индекса

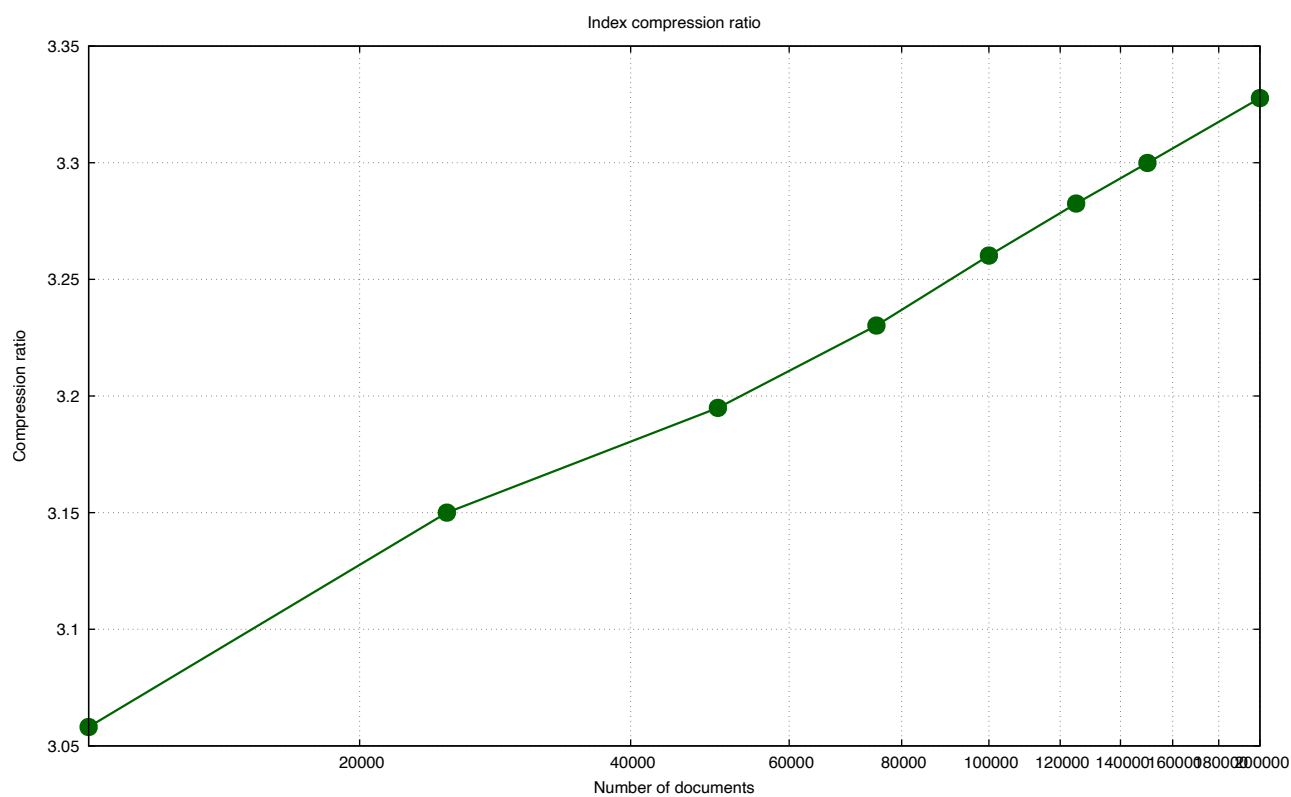


Рисунок 3.9 – Коэффициент сжатия в зависимости от числа документов

3.9 Профилирование

Для профилирования используется отдельная программа `cmd/profile/main.go`, которая выполняет построение индекса или поисковые запросы с включённым `pprof.StartCPUProfile`.

На рисунках 3.10 и 3.11 представлены графы вызовов CPU-профиля для построения индекса и поиска соответственно.

File: profile
Time: cpu
Duration: 303.09ms, Total samples = 90ms (29.69%)
Showing nodes accounting for 90ms, 100% of 90ms total
Showing top 50 nodes out of 58

See <https://git.io/JfYMW> for how to read the graph

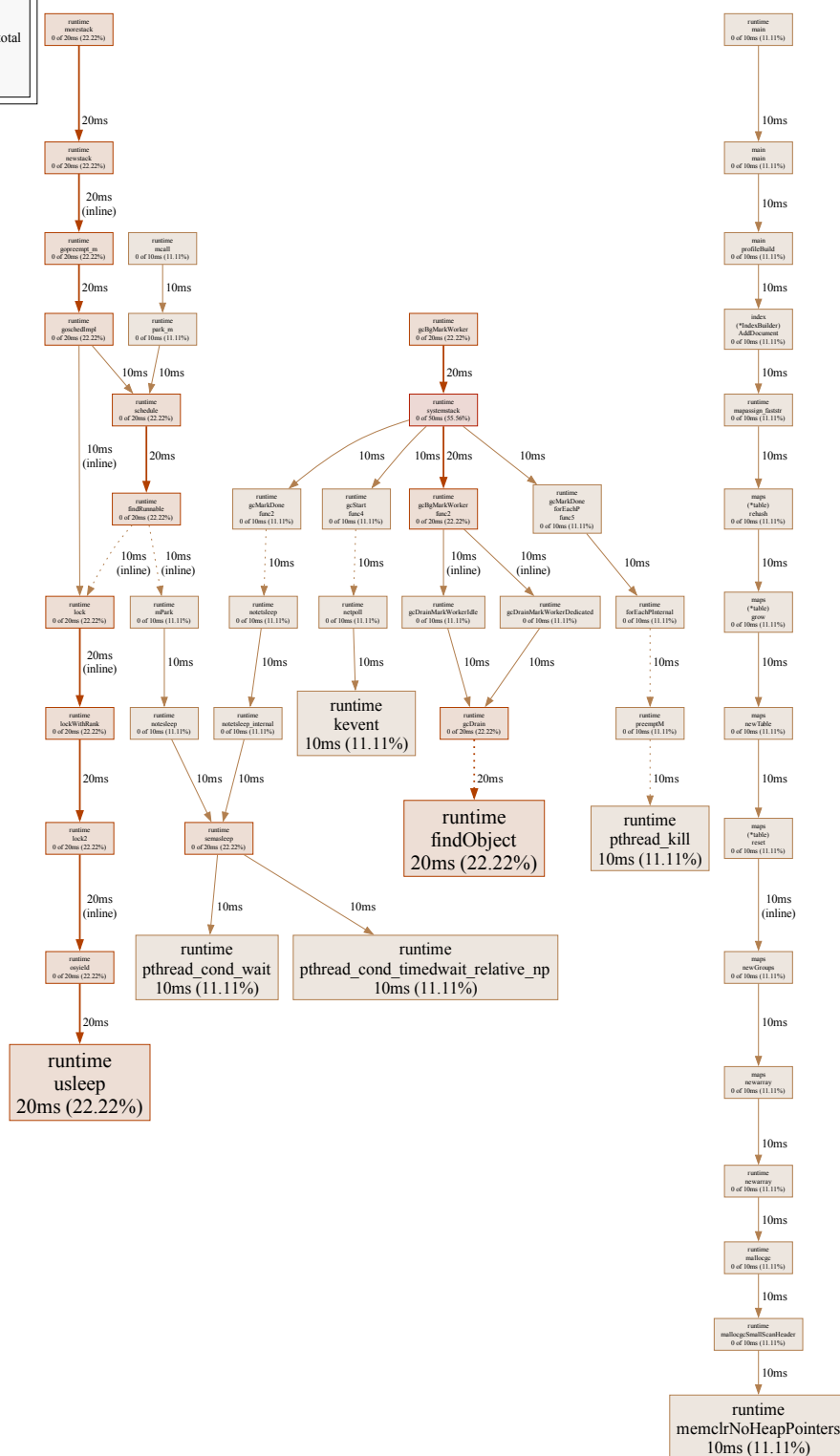


Рисунок 3.10 – CPU-профиль построения индекса

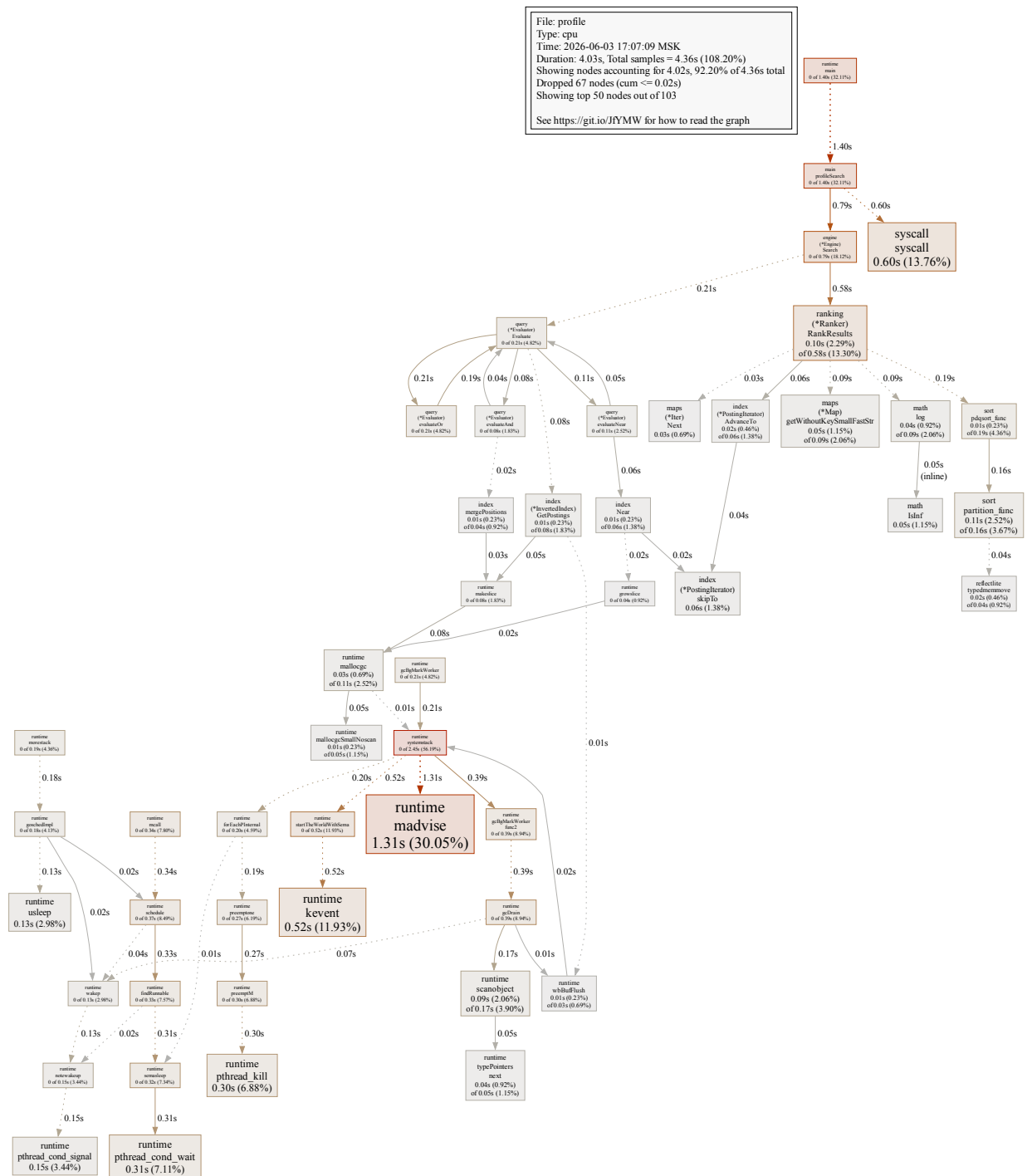


Рисунок 3.11 – CPU-профиль поиска

В профиле построения индекса основное время тратится на:

- `IndexBuilder.AddDocument` — токенизацию и обновление posting lists;
- `IndexBuilder.Save` — сортировку термов, битпакинг, построение skip lists и запись на диск.

В профиле поиска основное время приходится на:

- `InvertedIndex.GetPostings` — бинарный поиск термина и декодирование posting list;
- `Intersect/Union/Adjacent/Near` — операции над posting lists;
- `Ranker.RankResults` — вычисление BM25-оценок с использованием итераторов.

4 Вывод

Реализован обратный индекс с поддержкой поисковых операторов **AND**, **OR**, **NOT**, **ADJ**, **NEAR/*k***, ранжированием **BM25** и **TUI**-интерфейсом. Индекс хранится на диске в бинарном формате с битпакингом и дельта-кодированием **posting lists** и загружается через **mmap** с ленивым декодированием.

Проведены замеры производительности на наборах данных от 500 до 200000 документов. Можно сделать следующие выводы:

- время построения индекса растёт линейно;
- время **warmup** значительно превышает время повторного поиска;
- оператор **Term** — самый быстрый; **Or** — самый медленный;
- размер индекса растёт линейно.